Lab Report No _2_

Microprocessor and Interfacing techniques

Submitted By:

Huzaifa Shahbaz

21-CE-009

Submitted to:

Engr. Bushra Fiaz

Dated:

Week 02

Department of Computer Engineering,

HITEC University, Taxila

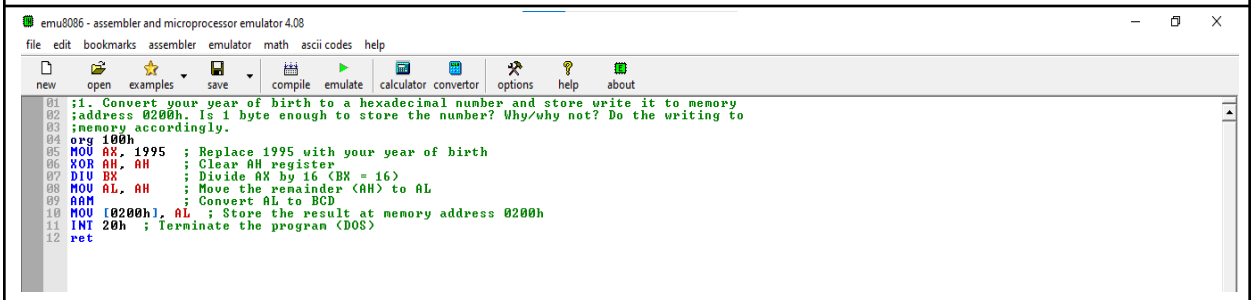
Lab Task No 1:

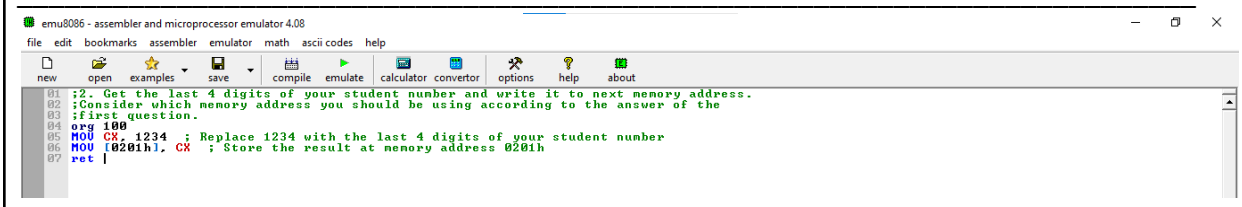
Solution:

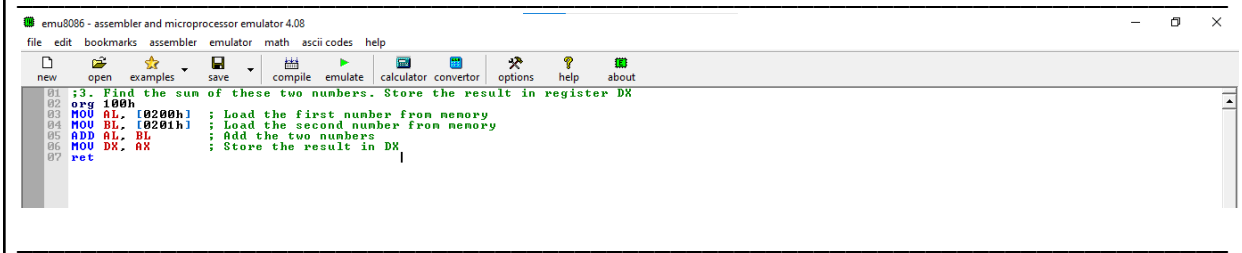
Brief description (3-5 lines)

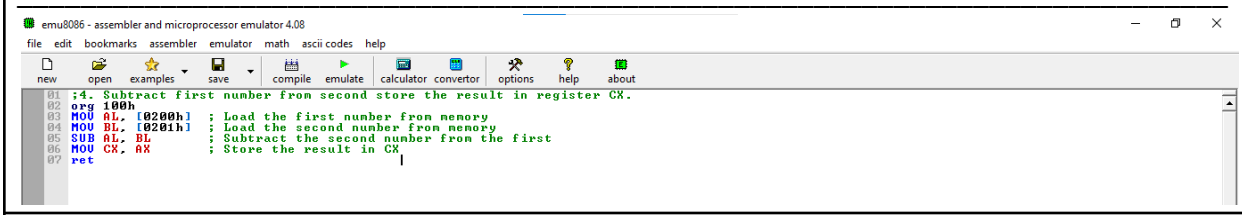
Direct Addressing and Arithmetic Operations. Write codes for the following
1. Convert your year of birth to a hexadecimal number and store write it to memory address 0200h. Is 1 byte enough to store the number? Why/why not? Do the writing to memory accordingly.
2. Get the last 4 digits of your student number and write it to next memory address. Consider which memory address you should be using according to the answer of the first question.
3. Find the sum of these two numbers. Store the result in register DX.
4. Subtract first number from second store the result in register CX.

The code

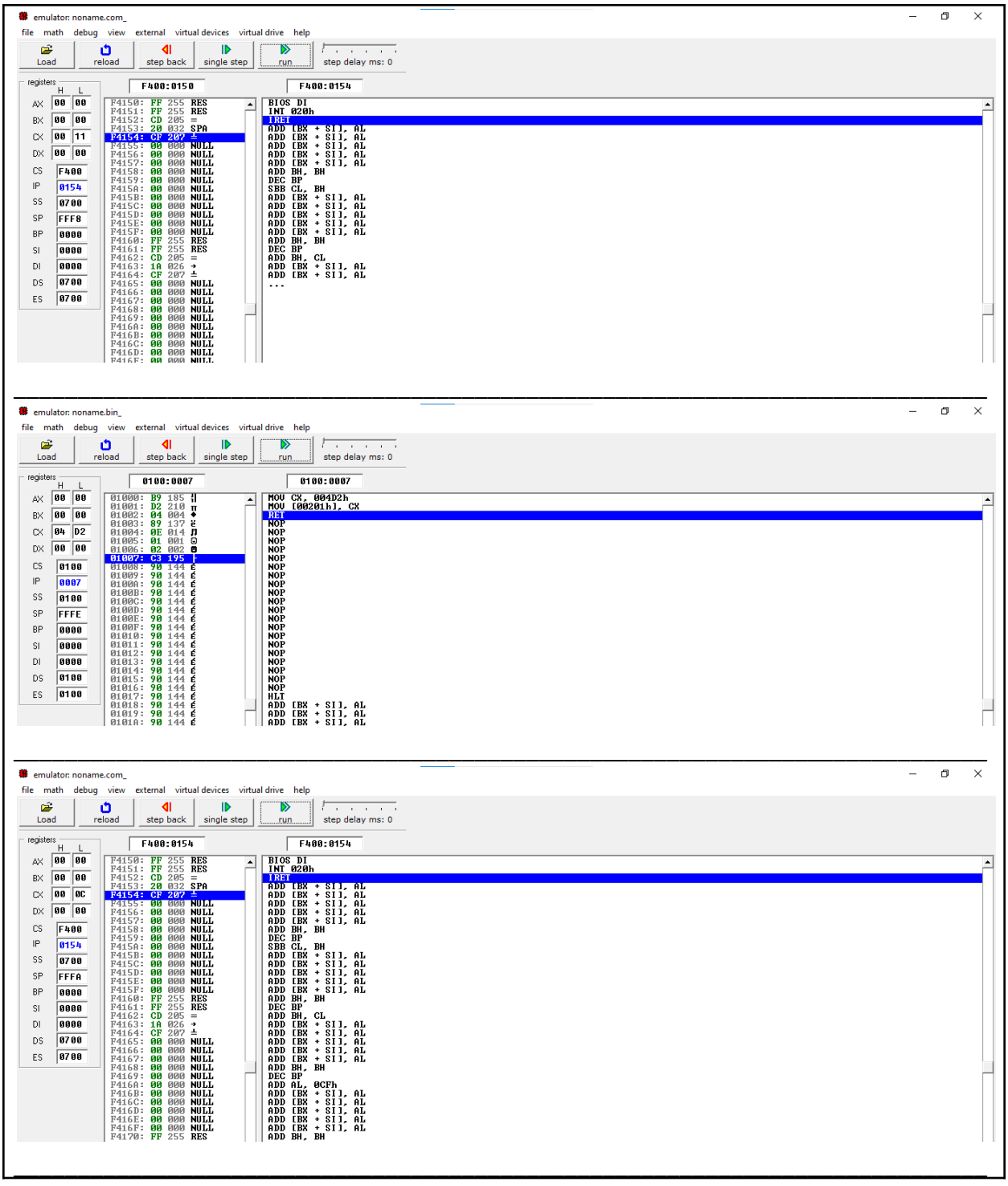


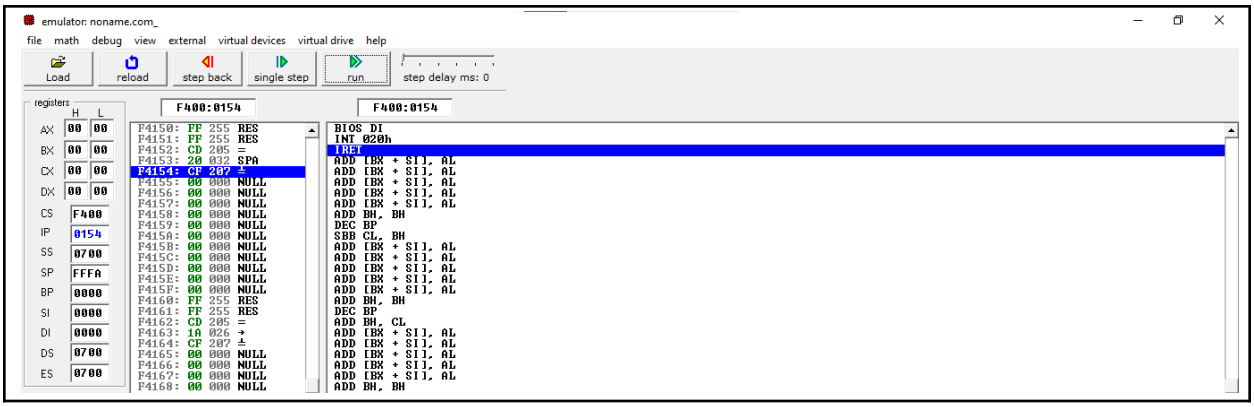






The results (Screenshot)


emulator: noname.com_
file math debug view external virtual devices virtual drive help
Load reload step back single step run step delay ms: 0
registers
F400:0150
F400:0154
emulator: noname.bin_
0100:0007
0100:0007
emulator: noname.com_
F400:0154
F400:0154



Lab Task No 2:

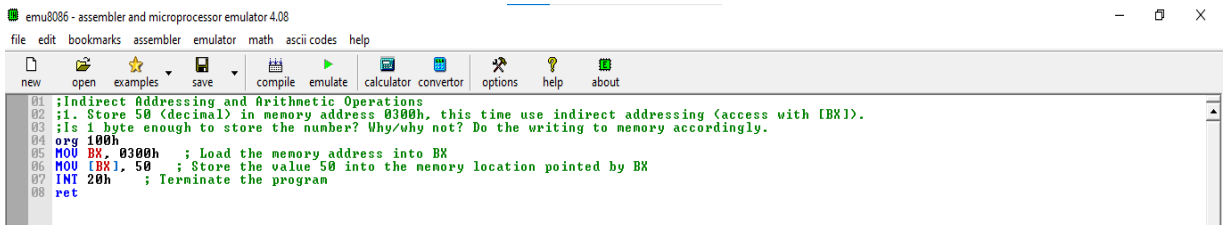
Solution:

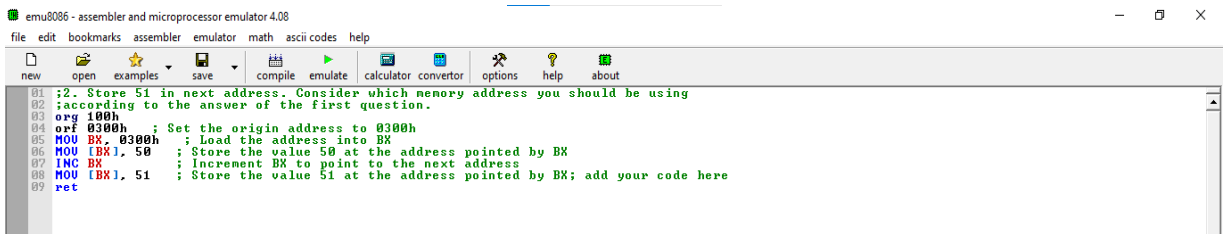
Brief description (3-5 lines)

Indirect Addressing and Arithmetic Operations

1. Store 50 (decimal) in memory address 0300h, this time use indirect addressing (access with [BX]). Is 1 byte enough to store the number? Why/why not? Do the writing to memory accordingly.
2. Store 51 in next address. Consider which memory address you should be using according to the answer of the first question.
3. Store 52 in next address.
4. Find the sum of 3 numbers. Store the result in the next address.

The code





emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
;3. Store 52 in next address.
org 100h
org 0300h    ; Set the origin address to 0300h
INC BX       ; Increment BX again to point to the next address
MOV [BX], 52   ; Store the value 52 at the address pointed by BX
ret
```

emu8086 - assembler and microprocessor emulator 4.08

file edit bookmarks assembler emulator math ascii codes help

new open examples save compile emulate calculator convertor options help about

```
;4. Find the sum of 3 numbers. Store the result in the next address.
org 100h
org 0300h    ; Set the origin address to 0300h
INC BX       ; Increment BX once more to point to the next address
MOV AL, [0300h] ; Load the value at address 0300h into AL
ADD AL, [0301h] ; Add the value at address 0301h to AL
ADD AL, [0302h] ; Add the value at address 0302h to AL
MOV [BX], AL    ; Store the result in the address pointed by BX
HLT             ; Halt the program
ret
```

The results (Screenshot)

emulator: noname.com_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers

	H	L
AX	00	00
BX	03	00
CX	00	09
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFF8	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

F400:0154

```
F4150: FF 255 RES
F4151: FF 255 RES
F4152: CD 205 =
F4153: 20 032 SPA
F4154: CF 207 ≟
F4155: 00 000 NULL
F4156: 00 000 NULL
F4157: 00 000 NULL
F4158: 00 000 NULL
F4159: 00 000 NULL
F415A: 00 000 NULL
F415B: 00 000 NULL
F415C: 00 000 NULL
F415D: 00 000 NULL
F415E: 00 000 NULL
F415F: 00 000 NULL
F4160: FF 255 RES
F4161: FF 255 RES
F4162: CD 205 =
F4163: 1A 026 →
F4164: CF 207 ≟
F4165: 00 000 NULL
F4166: 00 000 NULL
F4167: 00 000 NULL
F4168: 00 000 NULL
```

F400:0154

```
BIOS DI
INT 020h
IRET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
SBB CL, BH
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
ADD BH, CL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
```

emulator: noname.com_

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

registers

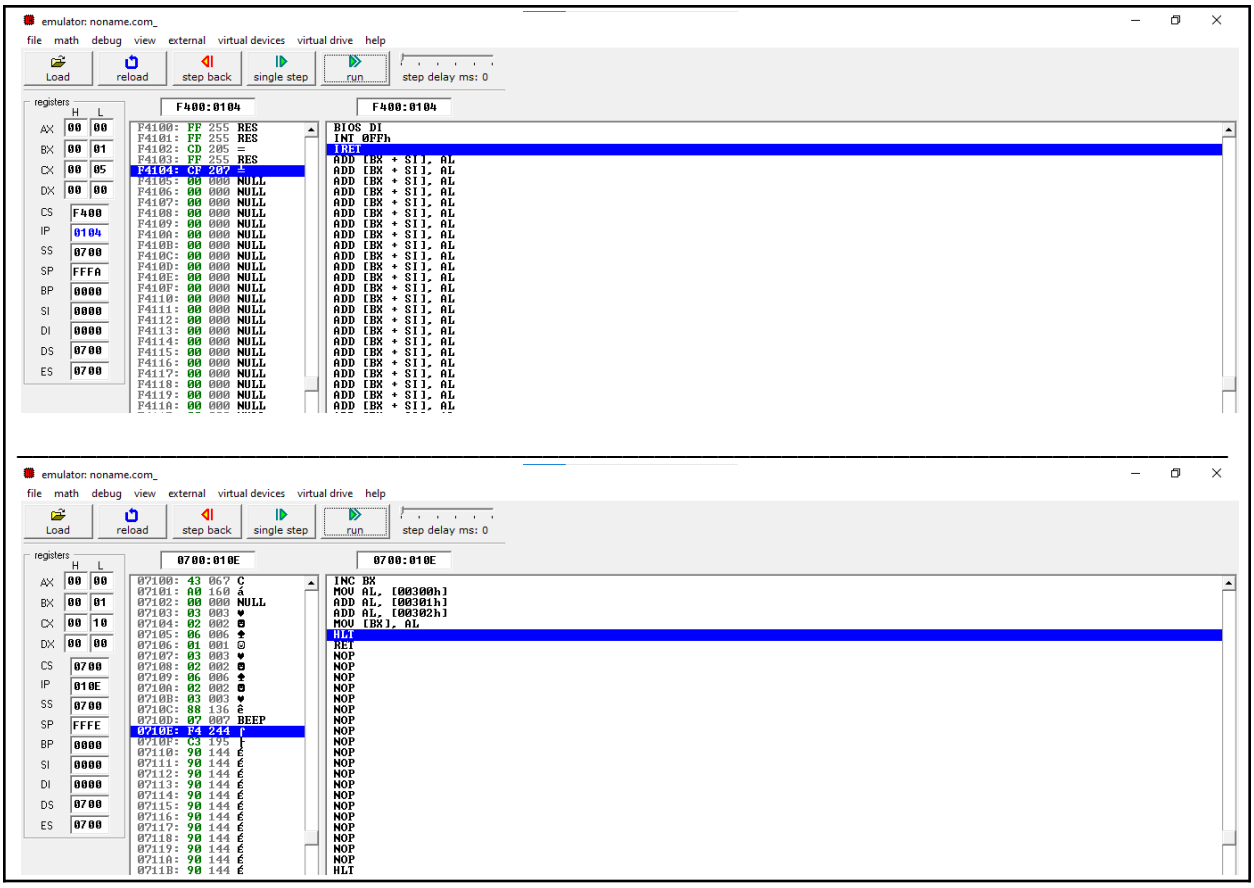
	H	L
AX	00	00
BX	03	01
CX	00	08
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

F400:0154

```
F4150: FF 255 RES
F4151: FF 255 RES
F4152: CD 205 =
F4153: 20 032 SPA
F4154: CF 207 ≟
F4155: 00 000 NULL
F4156: 00 000 NULL
F4157: 00 000 NULL
F4158: 00 000 NULL
F4159: 00 000 NULL
F415A: 00 000 NULL
F415B: 00 000 NULL
F415C: 00 000 NULL
F415D: 00 000 NULL
F415E: 00 000 NULL
F415F: 00 000 NULL
F4160: FF 255 RES
F4161: FF 255 RES
F4162: CD 205 =
F4163: 1A 026 →
F4164: CF 207 ≟
F4165: 00 000 NULL
F4166: 00 000 NULL
F4167: 00 000 NULL
```

F400:0154

```
BIOS DI
INT 020h
IRET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
SBB CL, BH
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
ADD BH, CL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
```



Lab Task No 3:

Solution:

Brief description (3-5 lines)
Advanced Data Transfer The program above prints “AB” to the console. The message “AB” is stored at msg in data segment. Therefore, “offset msg” is the starting address of the string. Modify the code according to the following experiments (Try both MOV and LEA commands): 1- Copy the second byte stored at “msg” (which is ‘B’ currently) to the first byte. The output should be “BB”. In order to do that, load 2-bytes data stored at “msg” into a 16-bits register. Copy one part of the register to the other part. Then, write your register back to msg. 2- Copy the first byte stored at “msg” (which is ‘A’ currently) to the second byte. The output should be “AA”. To achieve this, load the first byte stored at “msg” into one 8-bits register (high or low part) and copy it onto the other part and write the data stored in register back to msg.

3- Load the data stored at “msg” into one register and swap the high and low parts of the register using a temporary 8-bits register. Then write back your “swapped” register onto msg. The output should be “BA”.

The code

```
;Advanced Data Transfer
org 100h
.model small
.stack 1098
.data
msg db "AB", 0Dh, 0Ah, 156 ; Message with "AB", newline, and 'ó'
.code
main proc
    mov ax, @data
    mov ds, ax
    mov ah, 09h          ; Function code to display a string
    mov dx, offset msg   ; Load the offset of the message
    int 21h              ; Call DOS interrupt to display the message
    mov ah, 4Ch          ; Function code to exit the program
    int 21h              ; Call DOS interrupt to exit
main endp
end main
ret
```

The results (Screenshot)